

ARM: Autonomous Remediation and Management With LLM Agents for Intent-Driven Control

Vasilis Avgerinos¹, Kostas Ramantas², Luis Alonso³, and Christos Verikoukis⁴

Abstract—The growing complexity of cloud-native, edge, and internet of things (IoT) infrastructures has made manual configuration, fault remediation, and lifecycle management increasingly unsustainable. Traditional automation techniques—such as rule-based logic or bespoke machine learning pipelines—struggle with adaptability and explainability in dynamic environments. Recent advances in large language models (LLMs), however, have introduced new opportunities for autonomous, intent-driven infrastructure control. In this work, we present a closed-loop framework that integrates LLM agents for automated root cause analysis (RCA) and mitigation of faults within cloud-edge and IoT systems. When service-level agreement (SLA) violations are detected, the agent identifies likely root causes and selects corrective actions—such as pod rescheduling, scaling, or configuration updates—executed via a model context protocol (MCP) server exposing management tool functionalities through an application programming interface (API). This RCA-plus-mitigation loop enables fault handling that is both explainable and adaptive. We evaluate our system on a cluster running synthetic IoT workloads under emulated stressors using a reproducible benchmarking setup. Results show that the agent identifies SLA violations with 52.9% accuracy and mitigates 70.7% of them successfully. Notably, the agent incorporates validation steps to ensure system stability after interventions. These findings highlight the feasibility of LLMs for real-time infrastructure healing and their potential role in future artificial intelligence for IT operations (AIOps) workflows.

Index Terms—Agentic systems, artificial intelligence for IT operations (AIOps), autonomous infrastructure management, closed-loop control, edge computing, intent-based networking (IBN), internet of things (IoT) orchestration, Kubernetes, large language models (LLMs), service-level agreement (SLA) monitoring,

I. INTRODUCTION

THE growing complexity of deployed infrastructures—spanning telecom, cloud-edge, and internet of things (IoT) domains—has rendered manual configuration, fault mitigation, and lifecycle management increasingly unsustainable. Intent-based networking (IBN) emerged as a promising paradigm to address these issues by

enabling high-level, goal-driven management abstractions. However, for much of its existence, IBN remained more of a conceptual ideal than a deployable reality, held back by persistent challenges in translating natural language intent into actionable configurations and behaviors [1], [2]. While early efforts explored declarative approaches and query-driven summaries of network state, they often lacked the generalizability, context awareness, and autonomy required in large-scale, dynamic systems [3], [4]. As such, despite its potential, IBN's practical adoption stalled until recent years, where progress in AI-native orchestration, multiagent systems, and network softwarization began to make its core vision operationally viable [5].

Autonomous agentic systems—powered by recent advancements in large language models (LLMs) and small language models (SLMs), along with a rapidly expanding ecosystem of tools and techniques [6], [7]—have moved to the forefront of research and industry. A growing body of experiments demonstrates the increasing capabilities of LLM-driven agents across diverse domains: from solving novel mathematical problems and recommending bug fixes in production-grade codebases, to assisting in clinical diagnostics through application programming interface (API)-level interactions [8], [9], [10]. These developments suggest that we are approaching a level of agentic autonomy capable of transforming the long-standing vision of true zero-touch infrastructure management into a practical reality.

IoT infrastructures have long been a compelling testing ground for such systems. Their inherently heterogeneous, latency-sensitive, and dynamic nature makes them particularly well-positioned to benefit from these advances. As the number of connected devices continues to grow, so too does the demand for intelligent mechanisms capable of aligning low-level infrastructure behavior with high-level operational intents [11], [12], [13]. This alignment becomes especially critical when faults or performance anomalies risk violating service-level agreements (SLAs). In such cases, agentic systems can not only detect intent violations but also recommend or autonomously execute corrective actions—such as reconfiguring resources or rescheduling workloads—to restore compliance.

Traditional approaches to fault diagnosis in these environments have focused heavily on root cause analysis (RCA), typically using machine learning models trained on historical fault data. While effective in some scenarios, these models struggle to adapt to novel conditions and often lack explainability—two factors that undermine operator trust. Recent advances in Agentic AI suggest that LLM-based agents—such as ReAct [14]—can help address these short-

Received 17 July 2025; revised 25 November 2025; accepted 11 December 2025. Date of publication 26 December 2025; date of current version 27 April 2026. This work was supported in part by European Commission Research Projects AC3 under Grant 101093129 and in part by 6G-INTENSE under Grant 101139266. (Corresponding author: Vasilis Avgerinos.)

Vasilis Avgerinos is with the Industrial Systems Institute, 26504 Patras, Greece, and also with Iquadrat Informatica, 08006 Barcelona, Spain (e-mail: avgerinos@isi.gr).

Kostas Ramantas is with Iquadrat Informatica, 08006 Barcelona, Spain (e-mail: kramantas@iquadrat.com).

Luis Alonso is with the Universitat Politècnica de Catalunya, 08034 Barcelona, Spain (e-mail: luis.alonso@upc.edu).

Christos Verikoukis is with the Industrial Systems Institute, 26504 Patras, Greece, and also with the University of Patras, 26504 Patras, Greece (e-mail: chverik@gmail.com).

Digital Object Identifier 10.1109/IJOT.2025.3648858

2327-4662 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: Military Institute of Science and Technology. Downloaded on July 11, 2026 at 19:18:09 UTC from IEEE Xplore. Restrictions apply.

comings by coordinating external tools via natural language and producing transparent, traceable reasoning chains [15]. This is particularly relevant in IoT contexts, where the scale and volatility of services demand both rapid remediation and human-understandable diagnostics.

Based on the challenges and emerging opportunities outlined above, this article introduces **ARM—autonomous remediation and management** agent, leveraging the reasoning capabilities of LLM-based autonomous systems. Our approach seeks to bridge the gap between high-level operational intents and low-level reconfiguration actions by continuously monitoring for SLA violations and dynamically responding to deviations in system behavior. Users define custom intents with deployment- or application-level scope, which spawn dedicated monitoring loops that detect potential violations using Prometheus-based metrics pipelines. When an anomaly is detected—such as a performance degradation or fault condition—an LLM agent is triggered to investigate the underlying cause and execute an appropriate mitigation strategy. These strategies are implemented through an interface to a model context protocol (MCP) server [16], offering controlled access to tools like kubectl, observability modules, or autoscaling primitives. The agent operates in a closed-loop fashion, capable of interleaving planning, execution, and optional replanning phases until compliance is restored.

To validate our methodology, we simulate a range of representative fault scenarios under constrained resource conditions and assess the agent’s ability to recover, stabilize, and explain its interventions. The proposed architecture lays the groundwork for future research in integrating LLM agents into real-world artificial intelligence for IT operations (AIOps) pipelines, especially within latency-sensitive, decentralized IoT infrastructures. To the best of our knowledge, this is the first LLM-driven agentic system that not only identifies root causes of SLA violations but also autonomously mitigates them through infrastructure-level actions in cloud-edge and IoT environments.

Our key contributions are summarized as follows.

- 1) A novel closed-loop framework that integrates high-level intent specification with autonomous workload reconfiguration driven by LLM agents.
- 2) An LLM-powered control agent capable of investigating anomalies and executing infrastructure-level reconfiguration using tool calling and dynamic planning.
- 3) A reproducible evaluation cloud-edge environment based on a K3s Kubernetes cluster running synthetic IoT applications under emulated stressors.
- 4) A systematic evaluation of mitigation performance across multiple fault scenarios, demonstrating the feasibility and robustness of the proposed approach.

II. RELATED WORK

Achieving fully autonomous IoT-edge infrastructures depends on combining advancements from several key research domains. Our work is positioned at the intersection of IBN, natural language interfaces (NLIs), and runtime adaptation. This review outlines the current state of the art in these fields to describe the scientific landscape and identify the

specific research gaps our work aims to address. We observe that while each area has seen considerable progress, its holistic integration into a cohesive, closed-loop architecture remains an open challenge.

A. IBN for Autonomous Control

IBN has emerged as a foundational approach for automating network and service management, particularly within structured telco and cloud domains. Contributions from [17] and [18], for instance, showcase mature frameworks that translate abstract service requirements into concrete resource configurations for 5G network slicing and multicloud orchestration. A primary limitation of these systems, however, is that they are predominantly declarative and engineered for homogeneous environments, with an emphasis on initial deployment rather than continuous runtime adaptation.

Efforts to extend IBN to the heterogeneous edge and IoT continuum have been made, addressing domain-specific challenges such as RAN scheduling [19], vehicular mobility [20], and serverless computing [21]. Nonetheless, these approaches frequently lack a holistic, closed-loop architecture that integrates predictive foresight and natural language understanding. Consequently, they often remain reactive or episodic, falling short of the continuous, proactive self-management required by dynamic edge environments.

B. NLIs for System Configuration

A key challenge in making complex infrastructures more accessible is simplifying the human-to-system interface. To this end, research into NLIs has yielded promising results in automating configuration tasks. Systems such as LACE [22] translate free-form text into structured access control policies, while other works have demonstrated the use of NLIs to facilitate 5G service provisioning [23], generate infrastructure-as-code for chaos engineering experiments [24], or perform automated repair of security misconfigurations [25].

A notable limitation of these systems, however, is the static nature of their outputs. They typically execute a one-time translation from a natural language directive to an executable artifact without incorporating mechanisms for runtime feedback or autonomous correction. This “fire-and-forget” characteristic underscores a critical disconnect between the initial ease of configuration and the need for dynamic, self-managing operational control in response to changing conditions.

C. LLM Agents for Root Cause Analysis

Conceptual frameworks like ETSI ZSM have established the principle of the closed management loop for intent assurance, providing a foundational model for end-to-end automation [26]. The goal of such frameworks is to create self-healing systems where the entire incident lifecycle, from detection to mitigation, is handled autonomously. The general architecture and challenges of building such autonomous agents are well-documented in recent surveys, which outline core components like planning, memory, and action modules [27].

To incorporate high-level reasoning into runtime control, recent advances have explored the use of LLM-driven agents. These agents combine the language understanding and reasoning capabilities of LLMs with a structured interface to interact with external tools, environments, or APIs. By interpreting natural language prompts and generating action plans or decisions, they can autonomously perform tasks traditionally requiring human intervention. Highly specialized systems like Flow-of-Action, for example, have demonstrated robust performance in complex diagnostic tasks by using standard operating procedures (SOPs) to guide the agentic workflow [28]. This approach enhances the reliability of RCA by imposing expert-defined constraints on the LLM, thus reducing hallucinations and steering the diagnostic process toward a correct trajectory.

However, a significant limitation of the current state of the art is the overwhelming focus on problem identification rather than autonomous mitigation. The majority of recent work on AI agents for IT operations centers almost exclusively on anomaly detection and RCA. Numerous recently proposed frameworks follow this pattern, developing sophisticated agents that can decompose tasks, query knowledge bases, and iteratively explore system data to pinpoint the source of a fault [27], [28]. Even advanced multiagent frameworks engineered for “Autonomous CloudOps” concentrate on orchestrating agents to produce “accurate, reliable, and relevant insights” for human operators, rather than executing the mitigation itself [29]. Systems like FLASH are also designed primarily to improve the accuracy and efficiency of the diagnostic phase [30]. While these represent a major step forward in autonomous problem analysis, they do not execute active mitigation. Their final output is typically a root cause diagnosis intended for a human operator, not an automated remedial action. This leaves a critical gap in the closed-loop automation vision, as the crucial step of recovery still requires manual intervention, highlighting the need for future research to shift focus from just diagnosis to fully autonomous problem resolution.

III. FRAMEWORK

A. Cloud-Edge Infrastructure

We consider a distributed and heterogeneous computing environment that spans both centralized cloud nodes and geographically distributed edge resources. The infrastructure is formally defined as the union $\mathcal{N} = \mathcal{N}_{\text{cloud}} \cup \mathcal{N}_{\text{edge}}$, where each node n_i is associated with a capacity vector $\mathbf{r}_i = (c_i, m_i)$ describing its computational and memory capabilities. While cloud nodes typically offer high-performance compute and abundant memory, edge nodes are inherently constrained, often deployed in proximity to data sources to reduce latency and bandwidth demands.

B. Application and Service Model

Application workloads are structured as directed microservice graphs $\mathcal{A} = (\mathcal{S}, \mathcal{D})$, where vertices $\mathcal{S}$ denote the set of services and edges $\mathcal{D}$ capture interservice dependencies and communication flows. Services are deployed as replicated components across the infrastructure, subject to both

platform-specific constraints and locality awareness to satisfy latency-sensitive use cases. This enables the flexible placement of computation closer to where data are generated, without sacrificing centralized orchestration.

C. Performance Monitoring Framework

To support intelligent decision-making, the system incorporates continuous monitoring of runtime behavior. For each service s_i , a time-indexed vector of key performance indicators is maintained

$$\mathbf{M}_i(t) = \{l_{i,95}(t), q_{i,\text{success}}(t)\}. \quad (1)$$

Here, $l_{i,95}(t)$ reflects the 95th percentile response latency, while the metric $q_{i,\text{success}}(t)$ captures the ratio of successful requests. These signals are benchmarked against service-specific baseline profiles $\mathbf{B}_i$, established during periods of stable operation. Such baselines act as a reference for runtime anomaly detection.

D. Intent-Based Autonomous Intervention Model

To enable self-adaptive behavior, the system includes a lightweight autonomous agent capable of reactive intervention. Rather than relying on static threshold rules, the agent is activated by high-level performance violation intents—structured, context-rich alerts that encapsulate both metric deviations and system state. These intents allow the agent to engage in intelligent diagnosis and response actions tailored to the scenario at hand.

1) *Trigger and Intent Generation*: Intervention is initiated when the system detects deviations between live metrics and baseline profiles. In such events, a structured intent object $\mathcal{I}(t)$ is generated, designed to encode the operational snapshot needed for agent reasoning

$$\mathcal{I}(t) = \{\text{namespace, timestamp, } \mathbf{B}_i, \mathbf{M}_i(t), \mathcal{C}(t)\}. \quad (2)$$

The field $\mathcal{C}(t)$ refers to the current contextual snapshot of the system, including topology, deployments, and constraints. This representation decouples intent processing from low-level monitoring and equips the agent with a holistic view of the anomaly landscape.

2) *Agent Intervention and Recovery Evaluation*: Once triggered, the autonomous agent executes a bounded diagnostic and mitigation sequence $\mathcal{A}(t)$ guided by its reasoning model. Recovery efficacy is evaluated by remeasuring the monitored indicators post-intervention and comparing them to the original baseline $\mathbf{B}_i$. This process enables a closed feedback loop for assessing intervention quality and building long-term trust in autonomous decisions.

The complete high-level architecture of the framework can be seen in Fig. 1(a).

IV. LLM AGENT METHODOLOGY AND DESIGN

A. Core Loop Architecture

Our autonomous agent, depicted in Fig. 1(b), is designed around a closed-loop, round-based control paradigm that seamlessly integrates monitoring, reasoning, and corrective

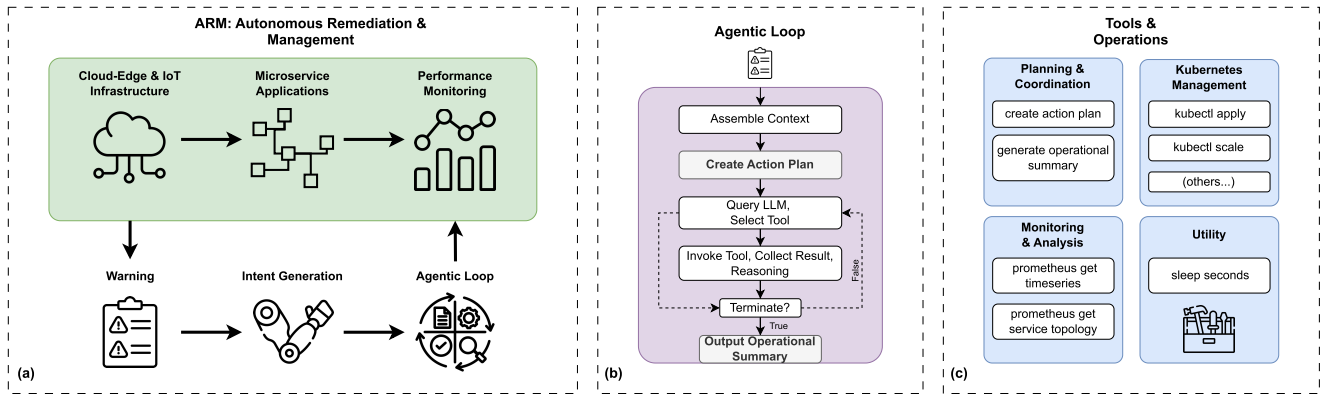


Fig. 1. Overview of the ARM system components. (a) End-to-end remediation flow from infrastructure monitoring to agent activation. (b) Internal agent loop for planning, tool invocation, and reasoning. (c) Structured tool layer categorized by functionality.

TABLE I
TOOL CATEGORIES AND REPRESENTATIVE OPERATIONS

Category	Representative Tools / Operations
Planning & Coordination	create_action_plan, generate_operational_summary
Kubernetes Management	kubectl_apply, kubectl_scale, kubectl_delete kubectl_drain
Monitoring & Analysis	prometheus_get_timeseries, prometheus_get_service_topology
Utility	sleep_seconds

actions. Influenced by the React framework [14], which extends chain-of-thought (CoT) prompting techniques [31] by interleaving reasoning and acting in a more interactive manner, the agent incrementally assembles a cumulative context at each iteration. This context includes the intent specification, system snapshots, outputs from previously invoked tools, and records of prior decisions. Importantly, this evolving context serves only as transient in-context information for the underlying model, and no parameter updates or external learning processes occur. The agent then queries the LLM, which selects the next tool to execute along with its parameters. To prevent redundant operations, only novel calls proceed to execution through temporal caching and recent-call filtering. The iterative process terminates when the LLM issues the `generate_operational_summary` call, or the maximum number of rounds is reached.

This architecture allows the agent to reason incrementally and adjust its behavior dynamically, while maintaining transparency. In contrast to multiagent or planner-executor frameworks, our single-agent loop is easier to audit and explain: each decision is informed by prior evidence, executed in a structured manner, and followed by immediate contextual integration of the result.

B. Tool Abstraction Layer

The agent's operational capabilities are encapsulated in a structured tool layer that provides safe, validated access to Kubernetes actions, metric collection, coordination primitives,

and planning operations. Tools are grouped into four categories shown in Table I, each exposing typed inputs and producing structured outputs that are parsed and integrated into the agent's evolving context.

Planning tools are central to the agent's reasoning. The `create_action_plan` tool encodes structured intent through fields such as a natural language objective, a list of explicit steps, and a set of risk-mitigation considerations. These fields provide a concrete plan trace that guides future decisions and helps external observers interpret the agent's rationale. At the end of execution, `generate_operational_summary` produces a detailed record of completed actions, observed results, and final status.

Kubernetes tools expose core resource management operations to the LLM, including scaling deployments, deleting pods, or inspecting system state. These tools operate through an intermediary MCP interface to ensure policy compliance and encapsulate error handling. By presenting these actions through abstracted calls (e.g., `kubectl_scale` with target namespace and desired replica count), the LLM can issue safe changes without low-level access. Additionally, safeguards and human approval workflows can be configured per tool to ensure that critical changes are carefully controlled and the system remains protected.

Monitoring tools retrieve time-series data from Prometheus using domain-specific filters. The most frequently used, `prometheus_get_timeseries`, allows queries over CPU, latency, error rate, and other key metrics. Important parameters include the duration (e.g., "2m") and the namespace or `service_name` to scope the analysis. These allow the agent to build temporal baselines, detect anomalies, and validate remediations. Other tools, such as `prometheus_get_service_topology`, expose dependency graphs that support causal reasoning during fault diagnosis.

Finally, utility tools such as `sleep_seconds` allow the agent to enforce stabilization delays after disruptive actions (e.g., pod deletions), ensuring that system metrics reflect steady-state behavior before proceeding.

Each tool call is followed by an immediate reasoning step in the loop. The agent records what was learned from the tool, how it affected the plan, and what should be done next. This

inline reflection mechanism is central to maintaining decision quality and justifiability.

C. System Prompt Philosophy

The behavior of the LLM agent is governed by a carefully engineered system prompt that blends structural constraints with heuristic guidance. This prompt defines the agent's operational role, enforces a disciplined planning-first lifecycle, embeds completion criteria, and encourages resolution-oriented behavior. Together, these mechanisms shape the agent's decision-making while preserving adaptability in dynamic infrastructure contexts.

1) *Role Definition and Behavioral Constraints*: At its core, the prompt frames the LLM as an *autonomous operator with the authority to execute corrective actions*. This role assignment is critical; it moves beyond passive diagnostics to an active, accountable execution model. The prompt encodes hard constraints that define legal agent behavior over the course of each loop. Three structural rules are enforced.

- 1) The first function call must always be `create_action_plan`.
- 2) If the agent's observations contradict its current assumptions, it must reinvoke `create_action_plan` to revise the strategy.
- 3) Termination is allowed only after a `generate_operational_summary` has been issued.

These rules ensure that every agent execution is bounded by an explicit plan and ends with a structured report. The planning call encodes an operational objective, a list of steps, and associated risks—enabling explainable reasoning and traceable execution. The final summary captures what actions were taken and why, offering post hoc transparency and accountability. Appendix presents an example planning-summary pair, demonstrating the structured reporting and traceability of agent actions.

2) *Evidence-Guided Reasoning and Completion Criteria*: To promote disciplined reasoning, the system prompt mandates an *evidence-first methodology*. The agent is not allowed to intervene until it has gathered sufficient data to justify the proposed action. This prevents speculative behavior and ensures that every step in the plan is grounded in observable system state.

The prompt also embeds an *action bias*, which discourages overanalysis. The agent is instructed to prioritize resolution over exhaustive diagnosis and to keep progressing toward remediation. In practice, this means that the agent continually collects evidence and reflects on its progress, but with the clear expectation that a fix—or a justified decision to not intervene—must eventually be produced.

To avoid premature termination, the system defines multipart completion criteria: execution can only end when: 1) system metrics trend toward baseline and 2) the agent provides a convincing rationale for inaction. These constraints ensure that the agent never halts in an ambiguous or unresolved state.

3) *Heuristic Guidance and Pattern-to-Action Mappings*: While structural rules enforce discipline, the prompt also

TABLE II
EXAMPLE EVIDENCE-TO-ACTION MAPPINGS

Evidence Pattern	Recommended Action	Suggested Tool
High CPU + High Latency	Scale out or increase CPU limit	<code>kubectl_scale</code> , <code>kubectl_patch</code>
Low CPU + High Latency	Analyze service call graph	<code>prometheus_get_service_topology</code>
Single Pod Anomaly	Restart the pod	<code>kubectl_delete</code>
Memory Growth + Restarts	Increase memory limit	<code>kubectl_patch</code>

provides soft heuristics to guide decision-making. Among these are *pattern-to-action mappings*—lookup tables embedded directly in the prompt that associate known symptom signatures with recommended interventions. For example, the pairing of high CPU usage and elevated latency suggests either scaling the workload or modifying resource limits. Table II offers a simplified extract of these mappings for illustrative purposes, noting that the full actionable logic embedded within the prompt is too extensive to be presented in its entirety.

These heuristics are not binding but serve as priors for tool selection. They accelerate convergence by steering the LLM toward common fixes while preserving flexibility for less canonical cases. This dual strategy—firm structural scaffolding and soft procedural suggestions—allows the agent to remain both predictable and adaptive, a critical balance in high-stakes infrastructure automation.

V. EXPERIMENTAL FRAMEWORK

A. Infrastructure and Cluster Setup

Our experimental framework is deployed on a virtualized cloud-edge cluster that emulates geographically distributed IoT environments. The infrastructure consists of seven nodes organized across three zones: two cloud nodes, four edge nodes (two per edge site), and one management node responsible for orchestration and observability. Each node runs as an independent virtual machine connected via a shared overlay network.

The cluster reflects realistic compute asymmetry between cloud and edge environments: cloud nodes are provisioned with 4 vCPUs and 4 GB of memory, while edge nodes operate with 2 vCPUs and 2 GB of memory. All nodes are pinned to dedicated host cores to ensure performance isolation and repeatability. Cluster management and workload scheduling are handled by a **k3s** Kubernetes distribution deployed on top of QEMU-virtualized servers. Prometheus and Grafana are integrated for metrics collection and visualization, while the control node coordinates chaos experiments, recovery actions, and data logging.

B. IoT Application Emulator and Scenarios

1) *Emulator*: To emulate realistic IoT workloads, we employ an extended version of the MuBench [32] microservice benchmarking framework. MuBench is used to generate

TABLE III
CHAOS FAULT TAXONOMY AND EXPERIMENT DISTRIBUTION

Scope	Mode	CPU Stress	Memory Stress	Pod Disruption	# Experiments	Notes
Pod-level anomalies						
Pod	one_pod	✓	✓	✓	6	Affects one pod per app
Pod	all_pods_in_app	✓	✓	✓	6	Affects all pods in app
Node-level anomalies						
Node	node_cpu_stress	✓	—	—	3	Host-wide CPU contention
Node	node_memory_stress	—	✓	—	3	Host-wide memory pressure
Control experiments						
Ctrl	baseline (no_failure)	—	—	—	2	Reference baseline runs
Total per application:						20

synthetic but semantically coherent microservice applications that mimic typical IoT service chains. The framework defines service dependencies, computational loads, and placement affinities that determine the deployment of microservices across the cloud–edge hierarchy.

Each application is specified via a `workmodel.json` configuration, which encodes the set of microservices, their interservice calls, and associated resource stressors (CPU, memory, disk, and network). This representation allows reproducible deployments of varying complexity—from lightweight gateway systems to multitier analytics pipelines—while maintaining consistent observability and fault injection capabilities across all experiments.

2) *Application Scenarios*: Three representative IoT applications are deployed using the emulator; the microservice graphs and deployment layouts appear in Fig. 2 (see the dedicated section/figure for full diagrams).

- 1) *IoT Gateway System (Three Microservices)*: Hybrid edge–cloud setup with most traffic processed at the edge for real-time device interaction and cloud handling complex analytics.
- 2) *Smart Manufacturing System (Eight Microservices)*: High-throughput, low-latency data processing focused on safety-critical operations and predictive maintenance.
- 3) *Smart City Traffic Management (12 Microservices)*: Multitier microservices with diverse latency and throughput needs, integrating real-time monitoring, emergency coordination, and analytics.

C. Chaos Engineering and Anomaly Injection

The fault injection campaign follows chaos engineering principles to systematically evaluate system resilience. Faults are injected at two primary scopes: 1) **service-level** (pod-level) anomalies, including CPU and memory stress and pod disruptions, which target one or all instances of a given microservice, and 2) **node-level** anomalies that simulate host-wide CPU or memory contention using the *chaosd* library. The complete taxonomy of fault modes and experiment counts is presented in Table III.

As shown in Table III, each experiment group covers both localized (pod-level) and system-wide (node-level) stressors, in addition to baseline control executions. The separation by scope emphasizes how different classes of anomalies affect

both microservice behavior and overall application latency. In total, each application undergoes 20 experiments, yielding 60 fault scenarios across all deployments.

D. Four-Phase Experimental Evaluation Protocol

Our evaluation framework employs a systematic four-phase protocol to assess autonomous agent performance under fault conditions:

Phase 1 (System Reset and Warmup): We reset the system by removing any existing node constraints and deploy a fresh application instance in the cluster. The system then initiates traffic generation and performs service warmup with latency stabilization detection. This ensures all microservices reach steady-state operation before baseline measurement.

Phase 2 (Baseline Collection): During stable operation, the framework captures comprehensive baseline vectors $\mathbf{B}_i$ for each service s_i , recording key indicators such as 95th percentile latency $l_{i,95}(t)$ and request success rate $q_{i,\text{success}}(t)$. These profiles are later used to determine whether recovery actions have restored normal conditions.

Phase 3 (Chaos Injection): Fault injection occurs while the system continues serving traffic, allowing real-time observation of performance degradation patterns. The deviation of runtime metrics $\mathbf{M}_i(t)$ from $\mathbf{B}_i$ also triggers the generation of structured alert intents $\mathcal{I}(t)$, initiating agent activation.

Phase 4 (Agent Recovery): Upon receiving the intent object $\mathcal{I}(t)$, the autonomous agent executes its diagnosis and remediation loop, continuously observing updated $\mathbf{M}_i(t)$ vectors to guide its tool selection. After execution concludes, we evaluate whether post-intervention metrics satisfy the success condition described in (3).

This four-phase design enables controlled evaluation of recovery latency, stability, and correctness under diverse anomalies, ensuring repeatability and comparability across all applications.

E. Experiment Success Criterion

We define a successful recovery as one where the agent brings each monitored performance indicator back close to its original baseline. Formally, for any metric $x_i \in \{l_{i,95}, q_{i,\text{success}}\}$ of service s_i , the recovery at time t' is considered successful if

$$\frac{|x_i(t') - x_i^{\text{baseline}}|}{x_i^{\text{baseline}}} \leq \epsilon_x \quad (3)$$

TABLE IV
AGENT PERFORMANCE COMPARISON BY FAULT TAXONOMY

Fault Mode	ID Acc. (%)		Success (%)		Lat. Rec. (%)		Rounds (avg min-max)		Time (min)	
	GPT-5	Mini	GPT-5	Mini	GPT-5	Mini	GPT-5	Mini	GPT-5	Mini
Control experiments										
Baseline (No Failure)	100.0	0.0	–	–	–	–	8.8 7–14	33.3 12–50	2.8	9.1
Node-level anomalies										
Node CPU Stress	100.0	88.9	55.6	44.4	80.4	79.4	18.8 11–36	19.7 0–35	9.1	6.0
Node Memory Stress	88.9	88.9	77.8	66.7	74	66	19.6 9–36	29.8 15–50	11.1	9.4
Pod-level anomalies										
Pod CPU (One Pod)	100.0	66.7	83.3	83.3	94.2	41.2	10.2 8–12	32.7 13–50	4.0	9.8
Pod CPU (All Pods)	83.3	83.3	83.3	33.3	82.4	97.9	23.0 8–36	31.7 17–50	13.6	9.5
Pod Failure (One Pod)	75.0	0.0	100.0	66.7	83.3	51.0	11.3 7–16	34.2 18–50	4.1	12.7
Pod Failure (All Pods)	66.7	66.7	83.3	83.3	100.0	83.3	14.5 7–22	27.7 19–39	6.1	9.0
Pod Memory (One Pod)	75.0	0.0	100.0	66.7	91.6	83.3	15.0 7–33	34.0 16–50	6.7	11.8
Pod Memory (All Pods)	50.0	0.0	83.3	100.0	65.2	57.1	14.3 8–36	30.7 10–50	10.2	10.5

where x_i^{baseline} is drawn from the baseline vector $\mathbf{B}_i$ and $x_i(t')$ is the corresponding post-recovery value. In our experiments, we instantiate the tolerances as

$$\begin{aligned}\epsilon_{l_{i,95}} &= 0.15 \quad (95\text{th percentile latency}) \\ \epsilon_{q_{i,\text{success}}} &= 0.01 \quad (\text{request success rate}).\end{aligned}\quad (4)$$

These metric-specific thresholds provide a unified benchmark across all applications and anomaly types, facilitating comparative evaluation of both recovery efficiency and correctness.

VI. RESULTS AND DISCUSSION

This section presents a comprehensive evaluation of two autonomous agents built on GPT-5 and GPT-5-mini, respectively, in identifying and mitigating system anomalies within the simulated cloud-edge environment. We analyze diagnostic accuracy, remediation success rates, decision rounds, wall-clock time, and tool utilization patterns, highlighting both the advantages of the larger model and the limitations that emerge for the smaller baseline in more complex scenarios.

All experiments utilize GPT-5 as the primary autonomous remediation agent and GPT-5-mini as a lightweight baseline, both operating over the same cloud-edge topologies, failure injections, and observability and actuation tools.

A. Agent Performance by Fault Type

Table IV summarizes performance across the fault taxonomy. Across control and node-level anomalies, GPT-5 achieves near-perfect fault identification, correctly recognizing the no-failure baseline and all node CPU stress cases (100.0% identification accuracy). GPT-5-mini fails to recognize the baseline as healthy (0.0%) and is slightly less reliable under node CPU stress (88.9% accuracy).

Remediation success for node CPU stress appears moderate for both agents (55.6% for GPT-5 and 44.4% for GPT-5-mini) when viewed through a binary success metric. However, this binary classification obscures the extent of the recovery. Under

our emulation regime, heavily constrained virtual machines often leave few viable scheduling or scaling options, meaning that a return to exact baseline levels is not always physically possible. To quantify the partial mitigation achieved, we introduce the latency recovery metric

$$\mathcal{R}_{\text{lat}} = \frac{l_{i,95}^{\text{chaos}} - l_{i,95}(t')}{l_{i,95}^{\text{chaos}} - l_{i,95}^{\text{base}}} \times 100 \quad (5)$$

where $l_{i,95}^{\text{chaos}}$ represents the peak 95th percentile latency observed during the fault injection, $l_{i,95}(t')$ is the post-recovery latency, and $l_{i,95}^{\text{base}}$ is the original baseline value.

Applying this metric reveals that despite the lower binary success rates, GPT-5 successfully reclaimed 80.4% of the performance degradation induced by CPU stress, while GPT-5-mini reclaimed 79.4%. This indicates that even when an optimal solution was infeasible due to resource exhaustion, the agents effectively identified the root cause and applied mitigations that substantially alleviated the system bottleneck.

At the pod level, GPT-5 sustains strong diagnostic performance, with identification accuracies between 50.0% and 100.0% across CPU, memory, and failure scenarios. GPT-5-mini, in contrast, struggles particularly with single-pod failures and memory anomalies, where identification can drop to 0.0% while still achieving moderate to high remediation success. GPT-5 also completes pod-level incidents in substantially fewer reasoning rounds (10.2–23.0 versus 27.7–34.2 rounds for GPT-5-mini) and typically in less wall-clock time.

An apparent discrepancy where remediation success exceeds identification accuracy is particularly pronounced for the smaller model. For example, in Pod Memory (All Pods) scenarios (Table IV), GPT-5-mini records 100.0% success despite 0.0% identification accuracy. This arises from how success is measured in our framework; it is defined purely in terms of the recovery of latency and success-rate metrics relative to the pre-fault baseline. In cases where the injected fault is subtle and fails to produce drastic observable degradation, the smaller model often fails to classify the specific root cause (resulting in 0% identification) but does not trigger erroneous actions that

TABLE V
AGENT PERFORMANCE BY APPLICATION

Application	ID Acc. (%)		Success (%)		Rounds (avg min-max)		Time (min)	
	GPT-5	Mini	GPT-5	Mini	GPT-5	Mini	GPT-5	Mini
IoT Gateway (3)	85.7	40.0	80.0	40.0	13.0 8-31	38.2 0-50	5.2	12.8
Smart Manufacturing (7)	87.5	66.7	90.0	80.0	14.8 7-33	25.8 10-48	6.2	8.5
Smart City (12)	80.0	64.7	80.0	75.0	18.6 7-36	25.5 12-50	11.8	7.4

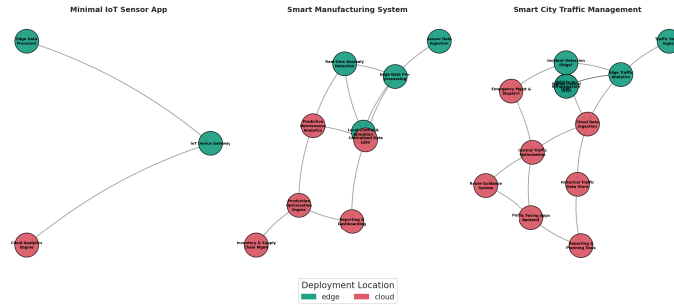


Fig. 2. Topologies of the three representative IoT applications, showcasing their microservice interconnections and deployment across edge and cloud environments.

worsen the state. Consequently, because the metrics remain within acceptable bounds, the run is algorithmically scored as a success. This highlights a limitation in the baseline model's sensitivity to latent faults compared to GPT-5, which maintains higher identification alignment.

B. Agent Performance by Application

Table V details the performance by application complexity. In the simpler IoT Gateway (three services), GPT-5 significantly outperforms the baseline (85.7% versus 40.0% identification accuracy) while requiring fewer decision rounds. As complexity increases to Smart Manufacturing (7 services) and Smart City (12 services), GPT-5 maintains robust reliability (identification accuracy >80% and success $\geq$ 80%). In contrast, GPT-5-mini struggles to scale, dropping to 64.7% identification accuracy in the largest topology and generally requiring more rounds to converge.

In terms of efficiency, GPT-5 typically completes remediations faster than GPT-5-mini (e.g., 5.2 versus 12.8 min for IoT Gateway), utilizing its capacity to plan more effectively. The exception is the Smart City topology, where GPT-5-mini exhibits lower resolution time (7.4 versus 11.8 min), likely reflecting shorter, less thorough reasoning trajectories. Overall, the larger model leverages its reasoning capabilities to achieve higher quality outcomes with comparable or better end-to-end latency.

The consistent stability of GPT-5 across varying topologies indicates that the model already possesses the requisite reasoning capacity to manage complex distributed systems. Consequently, effective scaling to broader enterprise-grade microservice architectures relies less on increasing model size and more on optimizing the information interface. Future developments should therefore prioritize intelligent context

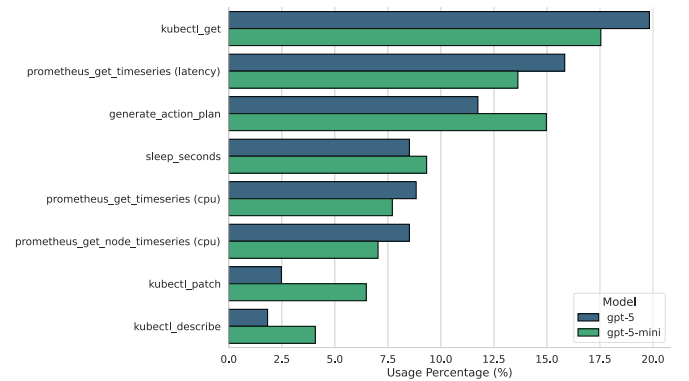


Fig. 3. Distribution of tool calls made by the agent during experiments. Observability tools (logs and metrics) dominate the interaction volume, reflecting the high cost of context gathering versus actuation.

compression and highly configurable tooling, enabling the agent to dynamically filter and prioritize vast telemetry data. This ensures that the model's reasoning potential is not bottlenecked by context limits, but rather empowered by a refined and efficient information architecture.

C. Tool Utilization Analysis

Tool usage statistics (Fig. 3) reveal distinct behavioral patterns despite a shared focus on observability. **GPT-5** exhibits a streamlined execution profile, utilizing `kubectl_apply` (3.77%) for decisive configuration updates while requiring fewer planning iterations (`generate_action_plan`: 11.75%). In contrast, **GPT-5-mini** displays a more fragmented workflow, relying heavily on granular debugging tools like `kubectl_describe` and `logs` (combined 7.6%) and invoking `generate_action_plan` significantly more

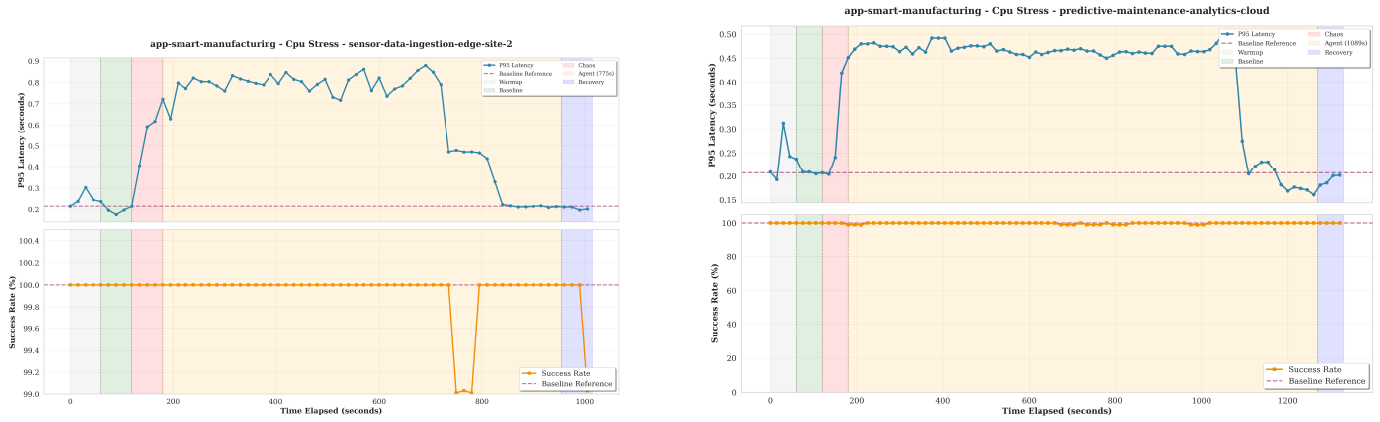


Fig. 4. Service metrics under CPU stress. Left: edge-based sensor data ingestion showing recovery after a disruptive intervention at $t \approx 750$ s. Right: cloud-based predictive maintenance analytics showing sustained analysis followed by effective mitigation at $t \approx 1100$ s.

often (14.97%). This increased reliance on replanning and localized patching (kubectl_patch: 6.48%) suggests that the smaller model often resorts to iterative trial-and-error, whereas the larger model acts with greater conviction and system-level awareness.

The relatively low frequency of actuation calls (e.g., scaling and restarting pods) compared to diagnostic calls indicates that the agent is cautious, preferring to gather sufficient evidence to rule out false positives. This ratio is critical for production environments, where the cost of a wrong action (such as restarting a healthy critical service) is high.

D. Agent Mitigation Dynamics

To illustrate the agent’s behavior across different infrastructure layers, Fig. 4 depicts the remediation trajectories for two distinct services within the Smart Manufacturing application: the edge-resident sensor-data-ingestion service and the cloud-hosted predictive-maintenance-analytics service.

Both traces exhibit a characteristic “diagnostic plateau” following the agent’s activation (indicated by the start of the yellow region). During this phase, the agent is actively querying observability tools to isolate the root cause from potential noise.

In the edge scenario (left), the persistence of high latency until $t \approx 750$ s suggests the agent first exhausted nondisruptive scaling options—which are naturally limited on edge nodes—before resorting to a pod restart. This decisive action is evidenced by the transient dip in success rate, which immediately precedes the return to baseline latency. Conversely, in the cloud scenario (right), the agent restores performance at $t \approx 1100$ s without incurring request failures, leveraging the greater resource elasticity available in the cloud tier to scale out the workload. These traces demonstrate the agent’s ability to adapt its remediation strategy to the specific constraints of the deployment environment, prioritizing stability while ultimately ensuring a return to optimal performance.

VII. CONCLUSION AND FUTURE WORK

In this work, we introduced a closed-loop framework for intent-driven infrastructure management that leverages LLM

agents to autonomously detect and remediate SLA violations in Kubernetes-based IoT and edge environments. Across three representative IoT applications, the GPT-5-based agent restores SLA metrics in roughly 80% of fault injection runs while correctly identifying the underlying condition in about 78% of cases, indicating that LLM agents can effectively close the loop in complex cloud–edge topologies. At the same time, the results reveal some important limitations, including nonnegligible wall-clock remediation times for the largest topology, and a distinct performance gap between model sizes, suggesting that the intricate causal reasoning required for truly autonomous, zero-touch control currently necessitates the advanced capabilities of frontier-class models.

Looking ahead, addressing scalability, safety, and integration with production IoT deployments will be particularly valuable. On the scalability front, we plan to investigate improved context-compression paradigms and deeper agent architectures equipped with a more diverse tool set spanning low-level metric queries to ML-based anomaly detection algorithms, as well as fine-tuning smaller models to better trade off accuracy versus model size in this domain. In parallel, we aim to formalize and evaluate safety safeguards around high-impact actions and to deploy the framework in larger, more heterogeneous IoT testbeds that more closely reflect real-world operational constraints.

APPENDIX

Initial Action Plan

Objective: Investigate and remediate performance degradation in namespace `app smart manufacturing` to restore latency and success rate to baseline.

Situation: Alert indicates p95 latency increased from 0.207 to 0.451 s (+ 118%) with success rate dropping from 100% to 99.25% as of 2025-11-13T09:29:21Z. Root cause unknown; need to confirm scope/severity and correlate with pod/node resource signals.

Steps:

- 1) *Phase 1—Scope and Severity:* Query 2-min time-series for latency, CPU, and memory in namespace `app smart manufacturing`; query node CPU

timeseries; get node readiness/health via `kubectl get nodes -o wide`.

- 2) *Phase 2—Pattern Analysis*: Correlate workload metrics with node metrics; list pods with placement via `kubectl get pods -o wide`; inspect service topology for upstream/downstream hotspots.
- 3) *Decision Branch*:
 - a) If `node_cpu` $\gg$ `pod_cpu` on a node with colocated pods showing high latency, cordon+drain that node.
 - b) If a specific pod shows runaway CPU/memory or restarts, delete/restart the pod.
 - c) If CPU saturation across replicas, patch HPA to raise min/max replicas appropriately.
 - d) If memory pressure is causing OOM/restarts, raise limits or scale out via HPA.
- 4) *Phase 3—Response and Validation*: Execute selected fix, wait 60 s, recheck latency/CPU, and iterate if needed.
- 5) *Completion*: When p95 latency trends back toward baseline and success rate $\geq 99.9\%$, produce operational summary with problem locations identified.

Expected Outcomes:

- 1) Quantified severity window and implicated resources identified (pods/nodes).
- 2) If node-level contention: affected node drained and workloads rescheduled.
- 3) If pod-level issue, problematic pod restarted and stabilized.
- 4) If a capacity issue, HPA scaled to add replicas, reducing per-pod CPU and latency.
- 5) Validated improvement with metrics trending toward baseline.

Operational Summary (Post-Execution)

Objective: Resolve performance degradation in namespace `app smart manufacturing` by mitigating CPU saturation and restoring latency toward baseline.

Status: COMPLETED—CPU saturation mitigated via HPA scale-out and affinity relaxation; latency trending toward baseline and node CPU normalized.

Actions Completed:

- 1) Collected 2-m namespace latency and CPU metrics; identified predictive-maintenance-analytics-cloud pods consuming ~3.3 cores each with cloud-worker-1/2 node CPU ~3.9 cores (saturation).
- 2) Verified HPA constraints causing `ScalingLimited`; patched predictive-maintenance-analytics-cloud-autoscaler to `minReplicas = 3` and `maxReplicas = 6`.
- 3) Validated limited improvement; inspected deployment affinity (restricted to zone = cloud).
- 4) Patched deployment predictive-maintenance-analytics-cloud nodeAffinity to allow zones [cloud, edge-site-1, edge-site-2] and restarted rollout to redistribute workload.

- 5) Revalidated metrics: pods rescheduled across edge and cloud nodes; node CPU normalized; service latencies dropped.

Results Achieved:

- 1) Predictive-maintenance-analytics-cloud replicas increased to 6 and distributed across cloud and edge nodes (observed new pods on edge-site-1/2 and cloud workers).
- 2) Cloud worker CPU pressure reduced (node `pod_cpu/host CPU` from ~3.4/3.9 cores down to ≤ 0.5 cores).
- 3) Latency improved across services: centralized data lake cloud ~450–480ms $\rightarrow$ 48–86ms; ingestion services reduced to ~97–287 ms, trending toward baseline 207 ms.

Problem Location:

- 1) Predictive-maintenance-analytics-cloud (deployment).
- 2) Predictive-maintenance-analytics-cloud-autoscaler (hpa).
- 3) Cloud-worker-1 (node).
- 4) Cloud-worker-2 (node).

REFERENCES

- [1] B. T. Loo et al., "Declarative networking," *Commun. ACM*, vol. 52, no. 11, pp. 87–95, Nov. 2009.
- [2] R. Birkner, D. Drachsler-Cohen, L. Vanbever, and M. Vechev, "Net2Text: Query-guided summarization of network forwarding behaviors," in *Proc. 15th USENIX Symp. Networked Syst. Design Implement. (NSDI 18)*, Apr. 2018, pp. 609–623. [Online]. Available: <https://www.usenix.org/conference/nsdi18/presentation/birkner>
- [3] A. Adewa, V. Anyah, O. David Olufemi, A. Ojo Oladejo, and T. Olaiya, "The impact of intent-based networking on network configuration management and security," *Global J. Eng. Technol. Adv.*, vol. 22, no. 1, pp. 63–68, Jan. 2025.
- [4] J. Kim, H. Okhravi, D. Tian, and B. E. Ujcich, "Security challenges of intent-based networking," *Commun. ACM*, vol. 67, no. 7, pp. 56–65, Jul. 2024.
- [5] S. T. Arzo, C. Naiga, F. Granelli, R. Bassoli, M. Devetsikiotis, and F. H. P. Fitzek, "A theoretical discussion and survey of network automation for IoT: Challenges and opportunity," *IEEE Internet Things J.*, vol. 8, no. 15, pp. 12021–12045, Aug. 2021.
- [6] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," 2020, *arXiv:2005.11401*.
- [7] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," in *Proc. Conf. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 68539–68551.
- [8] A. Novikov et al., "AlphaEvolve: A coding agent for scientific and algorithmic discovery," 2025, *arXiv:2506.13131*.
- [9] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "AgentCoder: Multi-agent-based code generation with iterative testing and optimisation," 2023, *arXiv:2312.13010*.
- [10] H. Zhang et al., "HuatuoGPT, towards taming language model to be a doctor," 2023, *arXiv:2305.15075*.
- [11] L. Bao, S. Yun, J. Lee, and T. Q. S. Quek, "LLM-hRIC: LLM-empowered hierarchical RAN intelligent control for O-RAN," 2025, *arXiv:2504.18062*.
- [12] S. Kou, C. Yang, and M. Wu, "SAFLA: Semantic-aware full lifecycle assurance designed for intent-driven networks," 2024, *arXiv:2404.12305*.
- [13] B. Adnan, S. Miryala, A. Sambu, K. Vaidhyanathan, M. De Sanctis, and R. Spalazzese, "Leveraging llms for dynamic IoT systems generation through mixed-initiative interaction," in *Proc. IEEE 22nd Int. Conf. Softw. Archit. Companion (ICSA-C)*, Mar. 2025, pp. 488–497.
- [14] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," 2022, *arXiv:2210.03629*.
- [15] A. Vitui and T.-H. Chen, "Empowering AIOps: Leveraging large language models for IT operations management," 2025, *arXiv:2501.12461*.

- [16] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model context protocol (MCP): Landscape, security threats, and future research directions," 2025, *arXiv:2503.23278*.
- [17] B. Martini, M. Gharbaoui, and P. Castoldi, "Intent-based network slicing for SDN vertical services with assurance: Context, design and preliminary experiments," *Future Gener. Comput. Syst.*, vol. 142, pp. 101–116, May 2023.
- [18] S. Braghin and L. Nedoshivina, "MIMO: A framework for extensible and flexible intent-based workload meta-orchestration," in *Proc. 2nd Int. Workshop MetaOS Cloud-Edge-IoT Continuum*, 2025, pp. 1–6.
- [19] S. Mostafa, M. P. Mota, A. Valcarce, and M. Bennis, "Intent-aware DRL-based NOMA uplink dynamic scheduler for IIoT," *IEEE Trans. Cognit. Commun. Netw.*, vol. 11, no. 6, pp. 4041–4049, Dec. 2025.
- [20] T. He, A. N. Toosi, N. Akbari, M. T. Islam, and M. A. Cheema, "An intent-based framework for vehicular edge computing," in *Proc. IEEE Int. Conf. Pervasive Comput. Commun. (PerCom)*, Mar. 2023, pp. 121–130.
- [21] N. Filinis et al., "Intent-driven orchestration of serverless applications in the computing continuum," *Future Gener. Comput. Syst.*, vol. 154, pp. 72–86, May 2024.
- [22] Y. Cheng et al., "Say what you mean: Natural language access control with large language models for Internet of Things," 2025, *arXiv:2505.23835*.
- [23] J. McNamara et al., "NLP powered intent based network management for private 5G networks," *IEEE Access*, vol. 11, pp. 36642–36657, 2023.
- [24] D. Kikuta, H. Ikeuchi, and K. Tajiri, "ChaosEater: Fully automating chaos engineering with large language models," 2025, *arXiv:2501.11107*.
- [25] Z. Ye, T. Huynh Minh Le, and M. Ali Babar, "LLMSecConfig: An LLM-based approach for fixing software container misconfigurations," 2025, *arXiv:2502.02009*.
- [26] K. Mehmood, K. Kravetska, and D. Palma, "Intent-driven autonomous network and service management in future cellular networks: A structured literature review," *Comput. Netw.*, vol. 220, Jan. 2023, Art. no. 109477.
- [27] D. Ziqi Zhou and M. Fokaefs, "AI assistants for incident lifecycle in a microservice environment: A systematic literature review," 2024, *arXiv:2410.04334*.
- [28] C. Pei et al., "Flow-of-action: SOP enhanced LLM-based multi-agent system for root cause analysis," in *Proc. ACM Web Conf.*, 2025, pp. 422–431.
- [29] K. Parthasarathy et al., "Engineering LLM powered multi-agent framework for autonomous CloudOps," 2025, *arXiv:2501.08243*.
- [30] X. Zhang et al., "Flash: A workflow automation agent for diagnosing recurring incidents," Microsoft Res., Redmond, WA, USA, Tech. Rep., 2024.
- [31] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," 2022, *arXiv:2201.11903*.
- [32] A. Detti, L. Funari, and L. Petrucci, " μ Bench: An open-source factory of benchmark microservice applications," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 3, pp. 968–980, Mar. 2023.